The Prompt Proposes, the Code Decides: Interface Design for Small Quantized Code Agents

Roberto Vetere · Independent researcher · github.com/RobertoVetere

Abstract

Small language models with 4-bit weights can run coding agents on modest hardware, but it is unclear how to connect them to tools. We study Gemma 4 E4B and 31B (4-bit weights) as agents that repair Python repositories under fixed limits, scored by hidden tests. The model proposes the change; deterministic scripts locate code, apply edits and run checks. We report four results of different strength. First, tool calls fail in five recurring ways; with an interface in which every call has a single field and long text goes through files, none of 105 calls was rejected, against 81–88% with the three-field call of earlier versions. Second, in an exploratory 2×2 experiment on 40 random tasks, reasoning added six solved tasks to a free-form agent and removed six from a script-guided one (sign test on the interaction, $p = 0.006$). Third, in exploratory single-call probes the quantized model generated a third as many tokens when reasoning and failed twice as many edits by miscopying code. Fourth, tuning tasks inflate success, runs do not repeat even at temperature 0, and development tasks can differ sharply from target tasks. We release the agents, a 104-task validation set, and tools to run the official evaluator outside the competition.

1. Introduction

Coding agents are usually designed for large models behind commercial APIs. A small model running locally with quantized weights writes tool calls less reliably, edits less precisely, and spends its time budget on the tokens it generates. Which design choices still hold in this setting is largely unmeasured.

We start from a simple principle: *the prompt proposes, the code decides*. The model chooses what to change. Deterministic scripts find the relevant code, apply the edit and run checks, and their verdict, not the model's own assessment, decides the next step. The principle comes from an earlier prompt-chaining tool in which validating each step raised a 0.5B model from 25% to 62.5% on 16 arithmetic problems (a single run).

We test the principle on repository repair: given an issue and a copy of the repository, the agent must produce a patch that passes hidden tests. Our measurements: a taxonomy of tool-call failures and an interface rule that removed them (4.1); an exploratory experiment on reasoning and agent structure (4.2); single-call probes of quantized and unquantized weights (4.3); a model-free retriever and an analysis of the remaining failures (4.4); and three evaluation pitfalls (4.5).

2. Experimental setup

Task. Each task is a Python repository at a fixed commit, an issue text, and hidden tests that the fix must make pass or keep passing. The agent runs in a sandbox without network, with limits of 8 minutes, 28 tool calls and 80 model turns.

Environment. We use the evaluation environment of the Gemma 4 Developer Agent competition [1]. Agents are declared in YAML on Google's Agent Development Kit (ADK) [2] and may use only nine predefined tools: shell command, read, write and edit file, submit patch, check status, and three queries about the code graph. The only custom code allowed is a set of Python scripts packaged as a *skill*, which the model invokes with a three-field call. The framework summarizes the conversation every 15 events, after which the model no longer sees its instructions. We run the competition's evaluator with one documented change, a sandbox patch so that repositories with a `src/` layout can be imported.

Models. Gemma 4 E4B and Gemma 4 31B with 4-bit weights and 16-bit activations (w4a16) [3], served with vLLM 0.19.1 [4] on four NVIDIA L4 GPUs with the Gemma 4 tool-call and reasoning parsers and a 32k context. Some experiments use the unquantized models through a commercial API (DeepInfra). Temperature is 0.2 unless stated otherwise.

Task sets. The *validation set* has 104 tasks from 24 repositories, from a public CC BY 4.0 collection [5], kept only if the reference patch passes the hidden tests in the competition environment (65 also run locally). The *development set* has the 72 locally runnable tasks of the competition's public data, from three repositories. The *competition test set* has 58 held-out tasks; we only see its aggregate score. Comparisons use the same tasks and are paired.

3. Method

Our agent, `fixer`, has three tools (`run_command`, `write_file`, `submit_patch`) and one skill. Its only skill call copies the scripts into a git-ignored folder in the repository; after that, every step is a one-field shell command. The E4B version follows eight steps:

1. install the scripts;
2. write the issue text to a file;
3. `evidence.py` lists candidate functions: those named in tracebacks, the entry point of any command quoted in the issue, and code that contains strings from the issue;
4. `show_function.py` shows a function and what it calls (at most two calls);
5. `callers.py` lists the places that call the function;
6. the model writes a small script of test cases, `check.py`;
7. the model writes a search-and-replace edit, which `apply_edit.py` applies, showing the closest real lines when the search text does not match;
8. `verify.py` runs the cases and a related existing test, and returns `SUBMIT`, `FIX` or `NO_EDIT`.

Every script ends with a `NEXT:` line that gives the exact next command, because the last tool output is always visible to the model, even after a summary. The 31B version is shorter: it skips steps 5 and 6, and `verify.py` instead runs the issue's own example before and after the change. In `joke2k__faker-2203`, the 31B fixed only the first failing line of the example; `verify.py` reported a new `AttributeError`, and the model completed the fix.

4. Experiments and results

4.1 Tool calls fail in predictable ways

We collected every rejected tool call in the traces of successive versions of our agents, run with the quantized models in the competition environment. Five failures recur. The first two come from framework conventions. The other three come from how the model writes a call when vLLM extracts it from free text; they did not appear against a commercial API that returns structured calls.

Failure	Cause	Remedy
The skill is called by the agent's name	The framework adds "your internal name is X" to every request	Give the agent and the skill the same name
The skill does not load	Only lowercase names with hyphens are accepted	Validate names before running
The call is written as text and never runs	The model imitated the code-like notation of the prompt	Describe each call as a list of fields
Fields are lost when an argument is long	The model leaves a string open and the parser merges fields	Pass long text through files
The same incomplete call is repeated 64 times	The summary removes the instructions	One-field commands and `NEXT` lines

Interface	Model	Task runs	Calls	Rejected for format	Edits not applied
Built-in tools (`edit_file`)	E4B	56	810	12 (1.5%)	57
Built-in tools	31B	16	459	3 (0.7%)	2
Skill, three-field call	31B	20	930	819 (88%)	0
Skill, three-field call, three prompt versions	E4B	3	43	12 (28%)	1
One-time install, one-field commands	E4B, 31B	8	105	0	4

Of the 819 rejected calls of the 31B with the three-field call, 653 were the naming failure. After names were fixed, the three-field call was still rejected in 130 of 160 calls (81%). With one-field commands, 0 of 105 calls were rejected (two-sided 95% upper bound: 3.5%). This compares successive versions, not randomized conditions, and the last row covers only eight task runs. With built-in tools, format failures are rare, but the small model loses edits because it does not reproduce the text to replace.

4.2 Reasoning interacts with the structure of the agent

We crossed two agents with two reasoning settings on 40 validation tasks sampled with a fixed seed, using unquantized E4B and one run per cell. The free-form agent is a public community agent from the competition, which edits files directly with the built-in tools. The script-guided agent is `fixer`.

E4B, 40 tasks	No reasoning	Reasoning (4,096 tokens)
Free-form agent	14 (67 s)	**20** (162 s)
`fixer`	**16** (63 s)	10 (150 s)

Tasks solved, with median seconds per task. 95% Wilson intervals [6] range from 14–40% (10 of 40) to 35–65% (20 of 40).

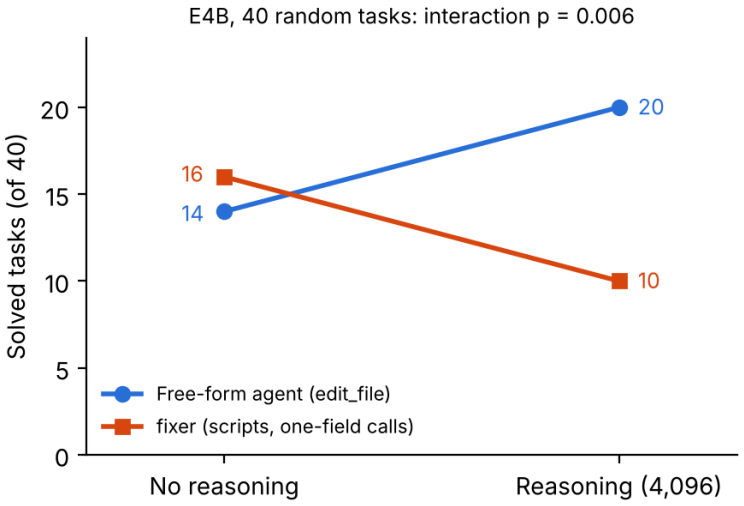


Figure 1: tasks solved out of 40 by agent and reasoning setting. Reasoning helps the free-form agent and hurts the script-guided one.

The two best cells are not significantly different: 5 tasks were solved only by the free-form agent and 1 only by `fixer` (exact McNemar test [7], p = 0.22). `fixer` was 2.5 times faster. Reasoning moved the two agents in opposite directions. For each task we computed the gain from reasoning of the free-form agent minus that of `fixer`; 11 tasks favour this interaction and 1 goes against it (sign test, p = 0.006). With reasoning, `fixer` reached the 80-turn limit on five tasks, and never without it. One reading is that, when scripts already fix the steps, extra thinking adds loops rather than better decisions. We treat this result as exploratory: there is one run per cell, the test was chosen after seeing the data, and the two agents differ in more than their structure.

Prompting did not improve the free-form agent. On 14 tasks where the agents disagreed, three prompt changes (reading whole functions, running the issue's example, "understand before editing") solved 8, 7 and 11, against 12 and 11 for the original in two runs; a fourth, a repository map, was never used. The 14 tasks were chosen with the original's results, which favours it.

4.3 What changes with quantization

To look at the model without the agent, we made single calls: one to locate the code to change from a repository map, and one to write an edit for that code, which we apply and score with the hidden tests. We ran the same prompts with the unquantized E4B (API) and the quantized E4B (our own vLLM 0.19.1 server) on 11 of the 14 disagreement tasks, two samples each (reasoning capped at 16,384 tokens).

Single call, E4B	**Unquantized**	**Quantized**
Fix, no reasoning	5 of 22	5 of 22
Fix, with reasoning	9 of 22	5 of 22
Median tokens generated per answer, with reasoning	6,002	2,002
Attempts lost because the search text did not match the code	11 of 88	24 of 88
Correct symbol located, 40 tasks: file list / repository map	28 / 31	24 / 29

Each model locates the code itself, and the located function differed in 6 of the 11 tasks, so the comparison mixes localization and repair; the two models were also served by different systems. With that caveat, the quantized model generated a third as many tokens when reasoning, and failed twice as many edits because the copied text did not match the code (Fisher's exact test, p = 0.02). The

difference in reasoning gain (9 against 5 of 22) is not significant ($p = 0.33$), and on the 10 samples where both models located the same code they solved the same number in every condition. We cannot attribute the missing reasoning gain to quantization.

Two observations inside the agent are consistent with this, as weak support. Raising the quantized agent's reasoning budget from 4,096 to 12,288 tokens changed nothing on 6 tasks (1 solved each): it wrote about 200 tokens per call. And in four runs on other task sets, the quantized free-form agent used a median of 6.5 to 18 tool calls per task, against 26 unquantized on the 40-task sample.

4.4 Where the remaining failures are

Localization is not the main problem. We built a retriever, `dossier.py`, that uses no model. It collects the functions executed by the issue's example, ranks function bodies with BM25 [8], matches identifiers from the issue, and adds the candidates' code, similar implementations in the same file and the existing tests that use them, each section preceded by an instruction (before the tests: passing them does not prove a fix). Compared with the reference patches, the changed function or its class is among the top three candidates in 28 of 40 validation tasks and 38 of 72 development tasks (the function itself: 23 and 28). In the single-call probes, when the quantized E4B had the right file, 55 of 64 repair attempts still failed.

The edit format is not the main problem either. We let the quantized model rewrite the whole function and replaced it by name in code. Copy failures disappeared, but solved attempts barely changed (6 against 5 of the 18 whose target was a function). What fails is the content of the change.

The larger model. A 31B agent that reads the retriever's report solved 4 of 8 development tasks in one run at temperature 0, and 1 of 4 and then 3 of 3 validation tasks in two runs at 0.2. The variance is high and the runs too small to compare with the smaller model.

4.5 Evaluation pitfalls

Tuning tasks inflate success. The 31B agent solved 7 of the 8 tasks used to design it, and 14 of 40 random tasks. Three of the eight are also in the 40.

Runs do not repeat. With temperature 0, top-p 1 and a fixed seed, 0 of 8 trajectories were identical across two runs on the same tasks; with identical inputs, the runs diverged at the third to fifth call. A likely cause is that batched inference kernels are not invariant to batch composition [9]; the agent's configuration cannot control it. A single run should be read as one sample.

Development tasks can differ from target tasks. The competition's tasks are pull-request descriptions rather than issue reports. Counting words of two or more letters, their median length is 72 words, against 108 in the 40 validation tasks; 23 of 72 have fewer than 30 words (2 of 40 in the validation tasks), and none contains an example that our extractor can run (7 of 40 in the validation tasks). Checks that run the issue's example, such as the 31B's `verify.py`, therefore never trigger on them. With unquantized weights, the small model solved 40–50% of the validation tasks. With the quantized weights, the same agents scored 0.01 and 0.03 on the competition test set, about one or two of 58 tasks. A Docker replica of the sandbox reproduced 4 of 5 validation results, so the environment does not explain the gap; shorter descriptions, quantized weights and different repositories cannot be separated with our data. A local task set compares versions; it does not predict the target score.

5. Related work

SWE-bench [10] defined repository repair scored by hidden tests; SWE-agent [11] showed that the agent–computer interface matters as much as the model. We study the opposite end of the scale: an interface that a small quantized model cannot call incorrectly. Another participant reports that

reasoning reduces malformed calls in small models [12]; we add that its value also depends on the agent's structure.

6. Limitations

Most comparisons use one run per condition on 40 tasks or fewer, so only large effects are detectable, and runs do not repeat exactly. The 2×2 experiment used unquantized weights and agents that differ in more than one respect; the probes cover 11 tasks without fixing localization; the tool-call results compare successive versions. All results come from one model family and one framework.

7. Conclusion

For small quantized models, the interface decides whether an agent works, not how much it solves. One-field calls with long text in files made tool use reliable, and scripts made localization and checking cheap; neither fixed the content of the change, where most remaining failures are. Because reasoning interacts with agent structure and quantization changes how the model reasons and edits, agent designs should be validated with the deployed weights, on random samples, with paired comparisons and repeated runs.

Reproducibility

Code, agents, data and per-task results are at https://github.com/RobertoVetere/nervo-gemma-paper (tag `paper-v1`). `docs/REPRODUCE.md` lists the file and command behind every number; the 2×2 table can be regenerated at no cost with `python3 local/tabla_lote.py resultados/local_deepinfra/l40`. The retriever of Section 4.4 and the 31B agents that use it will be added after the competition closes on 2 December 2026.

References

[1] E. Markowitz, B. Perozzi, B. Rózemberczki et al. *Google – The Gemma 4 Developer Agent Competition.* Kaggle, 2026. https://www.kaggle.com/competitions/gemma-4-developer-agent

[2] Google. *Agent Development Kit (ADK).* https://google.github.io/adk-docs/

[3] Google. *Gemma 4 quantization-aware trained checkpoints (w4a16).* Hugging Face, 2026.

[4] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023.

[5] D. Gluzdov. *gemma-4-task-soundness-and-local-results* (dataset, CC BY 4.0). Kaggle, 2026.

[6] E. B. Wilson. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 1927.

[7] Q. McNemar. Note on the Sampling Error of the Difference between Correlated Proportions or Percentages. *Psychometrika*, 1947.

[8] S. Robertson and H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 2009.

[9] H. He and Thinking Machines Lab. Defeating Nondeterminism in LLM Inference. *Thinking Machines Lab: Connectionism*, 2025.

[10] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024.

[11] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024.

[12] usp787. *Gemma4_paper_track.* GitHub, 2026. https://github.com/usp787/Gemma4_paper_track